



Algorithmic fundamentals



Complexité

Évaluer un algorithme (en taille et en temps)

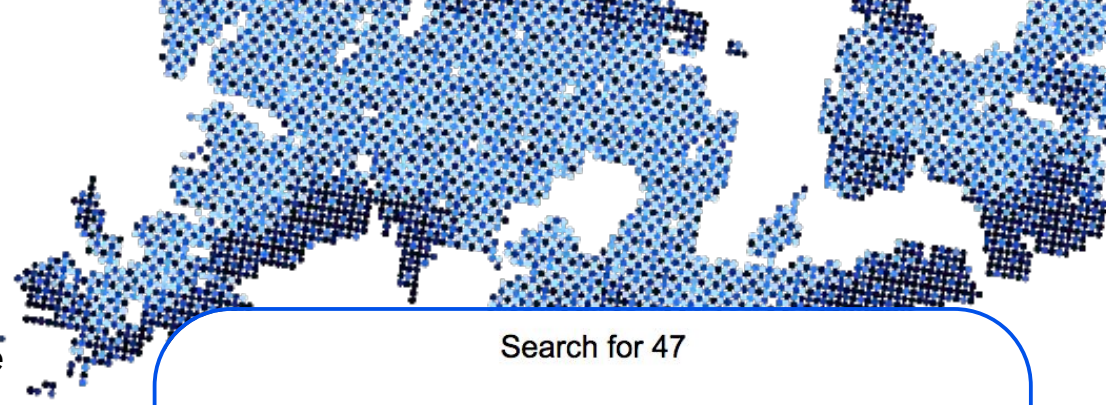
Commençons par un exemple...

Le contexte

Algorithme de recherche dans une liste triée

- Valeur cible T
- Liste triée $A = \{ A_1, A_2, A_3, \dots, A_n \}$

Objectif → avoir un algorithme qui détermine si T est présent dans la liste A



Search for 47

0	4	7	10	14	23	45	47	53
---	---	---	----	----	----	----	----	----

Commençons par un exemple...

Deux solutions

Recherche linéaire

```
● ● ●  
  
trouve = False  
i = 0  
while not trouve and i < len(A):  
    if(A[i] == T):  
        trouve = True  
    i += 1  
  
if trouve:  
    print("trouvé")  
else:  
    print("pas trouvé")
```

Linear search

Recherche dichotomique

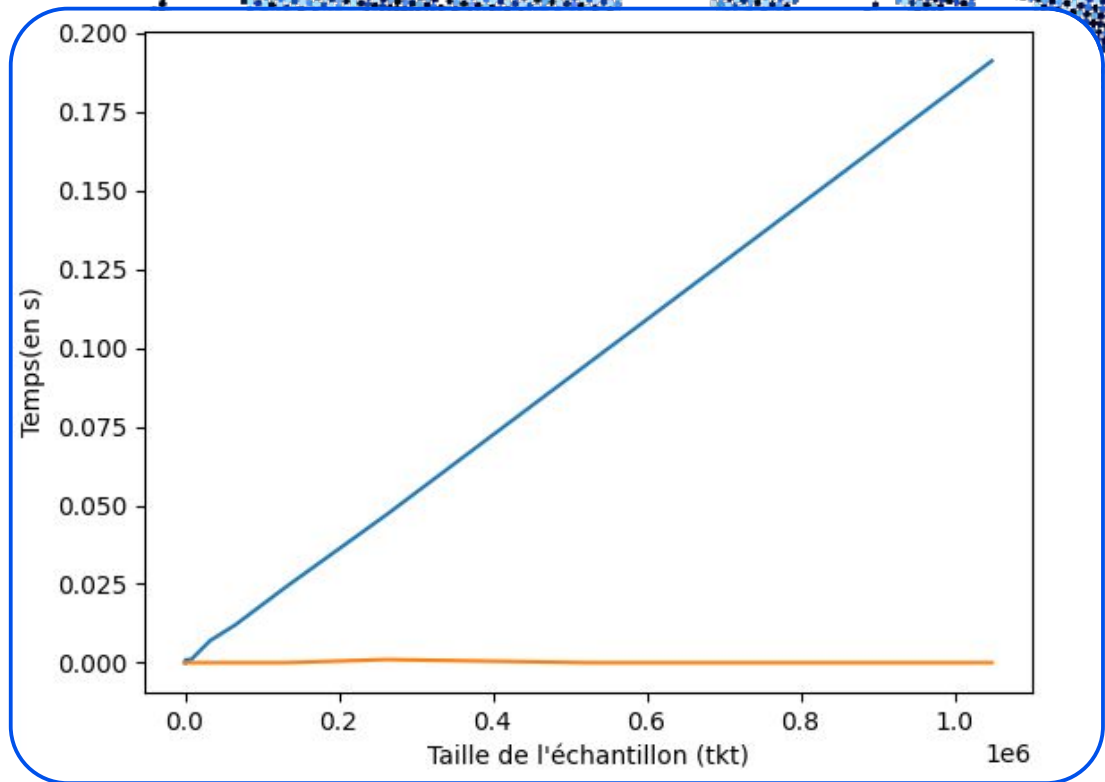
```
● ● ●  
  
start = 0  
end = len(A) - 1  
resultat = "pas trouvé"  
while (start <= end):  
    mid = (start + end) // 2  
    if(A[mid] > T):  
        end = mid - 1  
    elif(A[mid] < T):  
        start = mid + 1  
    else:  
        resultat = "trouvé"  
  
print(resultat)
```

Binary Search

Commençons par un exemple...

Comparons les deux algorithmes

- En **Bleu** : linear search
- En **Orange** : binary search



Commençons par un exemple...

Deux solutions

Recherche linéaire

```
● ● ●  
  
trouve = False  
i = 0  
while not trouve and i < len(A):  
    if(A[i] == T):  
        trouve = True  
    i += 1  
  
if trouve:  
    print("trouvé")  
else:  
    print("pas trouvé")
```

Boucle de taille N au pire cas
→ Complexité **linéaire** n

Recherche dichotomique

```
● ● ●  
  
start = 0  
end = len(A) - 1  
resultat = "pas trouvé"  
while (start <= end):  
    mid = (start + end) // 2  
    if(A[mid] > T):  
        end = mid - 1  
    elif(A[mid] < T):  
        start = mid + 1  
    else:  
        resultat = "trouvé"  
  
print(resultat)
```

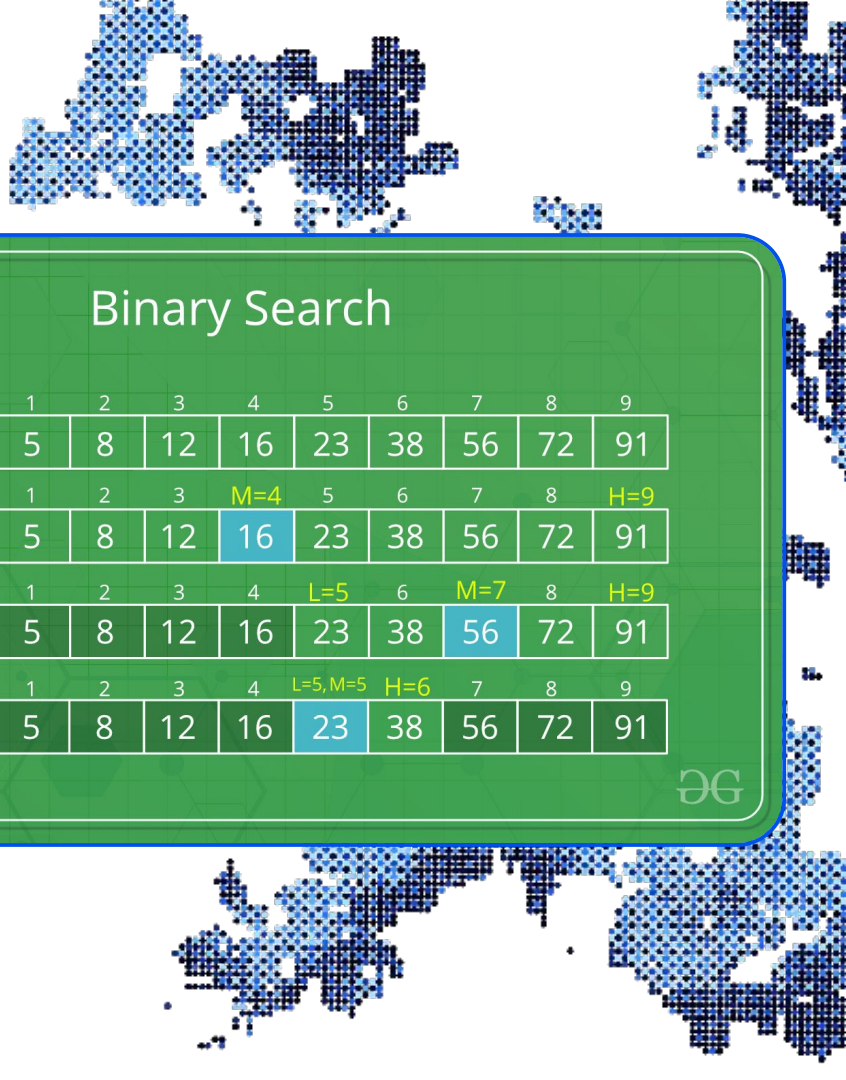
Quelle complexité ?

Commençons par un exemple...

À chaque étape, l'intervalle de recherche est divisé par deux
→ Complexité **logarithmique** $\log_2(n)$

Binary Search

	0	1	2	3	4	5	6	7	8	9
Search 23	2	5	8	12	16	23	38	56	72	91
	L=0				M=4					H=9
23 > 16 take 2 nd half	2	5	8	12	16	23	38	56	72	91
						L=5		M=7		H=9
23 > 56 take 1 st half	2	5	8	12	16	23	38	56	72	91
							L=5, M=5	H=6		
Found 23, Return 5	2	5	8	12	16	23	38	56	72	91



Une définition plus formelle

Nous avons évalué la complexité en **temps**

→ *Time complexity*

C'est à dire le temps que notre algorithme prend à s'exécuter en fonction de ses paramètres

Il est également possible de comparer l'**espace mémoire** utilisé par nos algorithmes

→ *Space complexity*

C'est à dire la taille de la mémoire que notre algorithme prends pour s'exécuter.



Une définition plus formelle

Lorsqu'on évalue une complexité, on utilise la notation $O(f)$ appelée "Grand O"

→ *Big O notation*

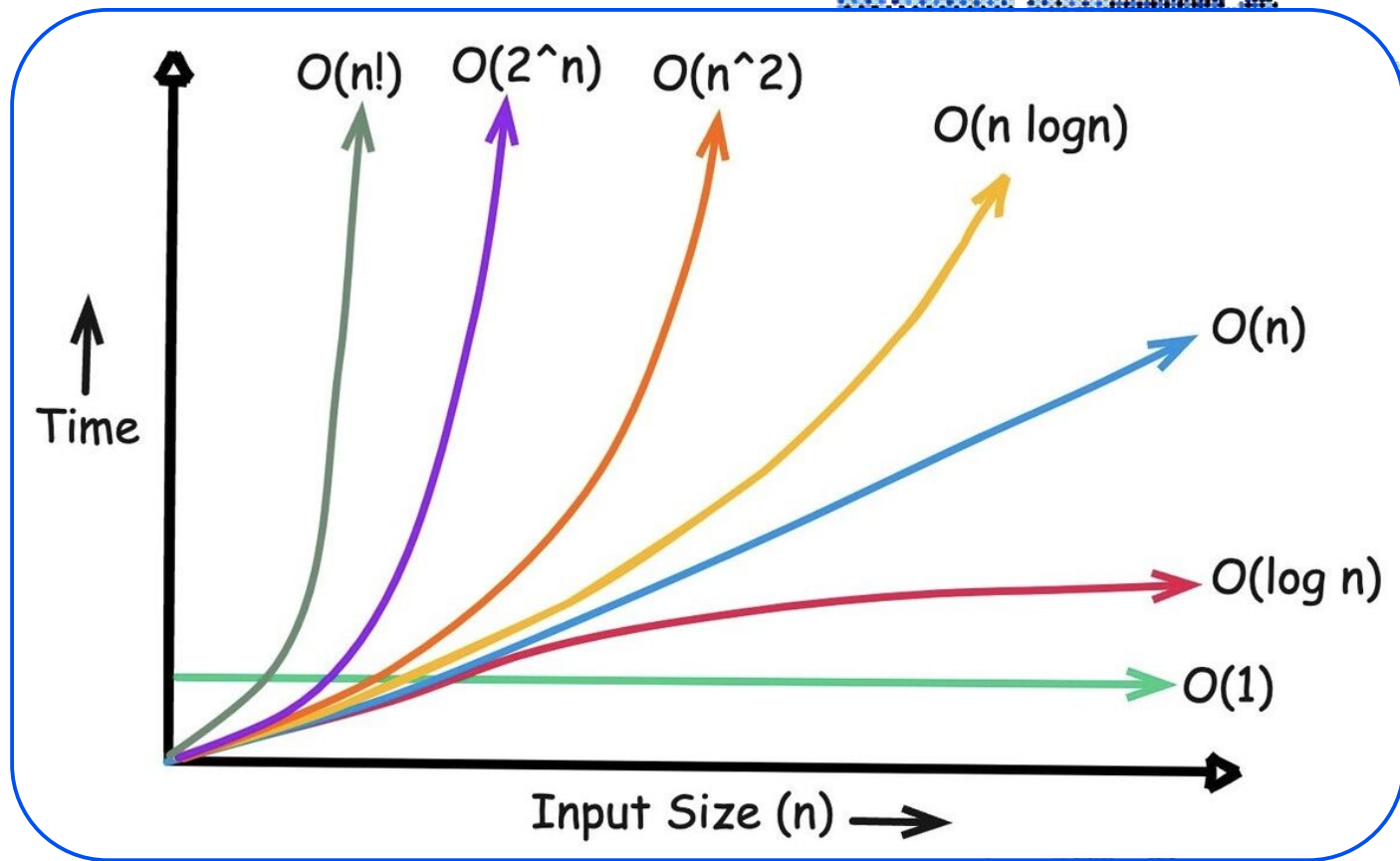
Quelques algorithmes :

- $O(1)$ Somme des nombres de 1 à n
- $O(\log(n))$ Recherche dichotomique
- $O(n)$ Explorer tous les éléments d'une liste
- $O(n \log(n))$ Tri d'une liste (merge sort)
- $O(n^2)$ Trouver l'ensemble des couples d'une liste
- $O(n!)$ Trouver l'ensemble des permutations d'une liste

etc..



Une définition plus formelle



Une complexité variable..

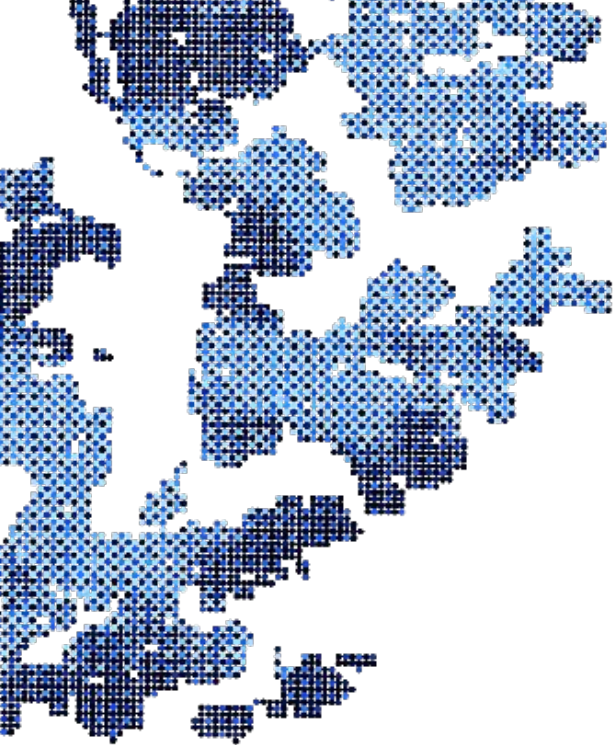
Parfois, la complexité algorithmique dépend de l'entrée (même d'une taille identique)

Par exemple, le tri rapide (*quick sort*) :

- $O(n \log(n))$ en moyenne
- $O(n^2)$ dans le pire cas

Lorsque l'on évalue la complexité d'un algorithme, on prends systématiquement le **pire cas**.





Array Sorting Algorithms

Algorithm	Time Complexity			Space Complexity
	Best	Average	Worst	Worst
<u>Quicksort</u>	$\Omega(n \log(n))$	$\theta(n \log(n))$	$O(n^2)$	$O(\log(n))$
<u>Mergesort</u>	$\Omega(n \log(n))$	$\theta(n \log(n))$	$O(n \log(n))$	$O(n)$
<u>Timsort</u>	$\Omega(n)$	$\theta(n \log(n))$	$O(n \log(n))$	$O(n)$
<u>Heapsort</u>	$\Omega(n \log(n))$	$\theta(n \log(n))$	$O(n \log(n))$	$O(1)$
<u>Bubble Sort</u>	$\Omega(n)$	$\theta(n^2)$	$O(n^2)$	$O(1)$
<u>Insertion Sort</u>	$\Omega(n)$	$\theta(n^2)$	$O(n^2)$	$O(1)$
<u>Selection Sort</u>	$\Omega(n^2)$	$\theta(n^2)$	$O(n^2)$	$O(1)$
<u>Tree Sort</u>	$\Omega(n \log(n))$	$\theta(n \log(n))$	$O(n^2)$	$O(n)$
<u>Shell Sort</u>	$\Omega(n \log(n))$	$\theta(n(\log(n))^2)$	$O(n(\log(n))^2)$	$O(1)$
<u>Bucket Sort</u>	$\Omega(n+k)$	$\theta(n+k)$	$O(n^2)$	$O(n)$
<u>Radix Sort</u>	$\Omega(nk)$	$\theta(nk)$	$O(nk)$	$O(n+k)$
<u>Counting Sort</u>	$\Omega(n+k)$	$\theta(n+k)$	$O(n+k)$	$O(k)$
<u>Cubesort</u>	$\Omega(n)$	$\theta(n \log(n))$	$O(n \log(n))$	$O(n)$

Quelques complexités
d'algorithmes de tri

Conversion complexité - temps d'exécution

Temps total = nombre d'opération / opérations par seconde

En Python..

Temps total = nombre d'opération / Entre 10^7 et 10^8

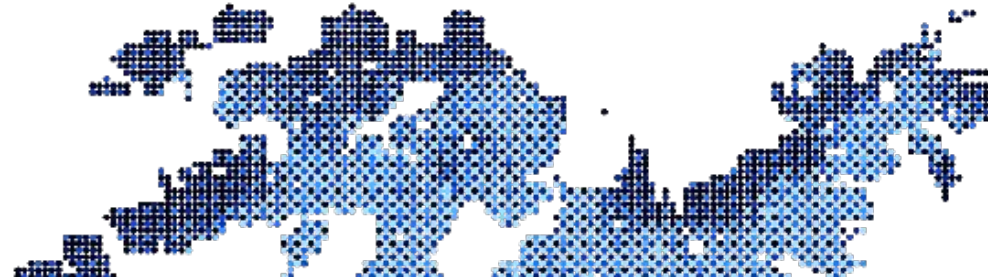


Pourquoi c'est utile ?

Contraintes du problème

- $0 \leq N \leq 10^2$
- $0 \leq C \leq 10^3$
- Time limit = 1s

Complexité	Nombre d'opération	Temps d'exécution
$N * C$	10^5	0.001s
$N^2 * C$	10^7	0.1s
$N * C^2$	10^8	1s
$N! * C$	$\infty?$	Non



Attention aux pièges !

Lors d'appels à des méthodes/librairies



```
array = ACollectionImplementation()  
targets = [1,2,5,6]  
for element in targets:  
    if(array.find(element)):  ← On ne connaît pas la complexité de l'algorithme  
        print(element,"was found")  
    else:  
        print(element,"was not found")
```

← On ne connaît pas la complexité de l'algorithme
derrière la méthode *found*



Espace de recherche et brute-force



Espace de recherche

Définition

Il s'agit de l'**ensemble des solutions possibles** à un problème donné

Exemple : trouver la paire avec la somme la plus petite

0 5 8 6 -1 4 9 10 2

→ Ici, l'espace de recherche est l'ensemble des paires



Espace de recherche

Exemple

Représentation des paires possibles

Matrice avec :

- Ligne : premier élément de la paire
- Colonne : deuxième élément de la paire

→ Implémentation : deux boucles qui parcourent la liste de manière imbriquée
→ Complexité algorithmique de $O(n^2)$

	0	5	8	6
0	/	5	8	6
5	5	/	13	11
8	8	13	/	14
6	6	11	14	/

Brute-force

Introduction

Exemple : je veux trouver tous les sous-ensembles d'un ensemble, dont la somme est égal à 42

Alors, je peux :

1. Créer une liste de tous les sous-ensembles
2. Pour chaque sous-ensemble, calculer la somme et tester si elle vaut bien 42



Brute-force Implémentation

La librairie *itertools* propose plusieurs outils pour construire un espace de recherche :

- `list(product("ab", "cd"))`
 - `[('a', 'c'), ('a', 'd'), ('b', 'c'), ('b', 'd')]`
- `list(permutations("abc", 2))`
 - `[('a', 'b'), ('a', 'c'), ('b', 'a'), ('b', 'c'), ('c', 'a'), ('c', 'b')]`
- `list(combinations("abc", 2))`
 - `[('a', 'b'), ('a', 'c'), ('b', 'c')]`
- `list(combinations_with_replacement("abc", 2))`
 - `[('a', 'a'), ('a', 'b'), ('a', 'c'), ('b', 'b'), ('b', 'c'), ('c', 'c')]`

→ Note : on utilise `list()` car les valeurs de retour sont des itérateurs



Brute-force

Réolvons un problème !

Réolvons notre problème

Sortie :

- (12, 30)
- (25, 17)
- (25, 8, 9)
- (10, 12, 14, 6)
- (10, 17, 9, 6)



```
from itertools import combinations

arr = [10, 12, 25, 30, 17, 8, 14, 9, 6]

gen = (sub for size in range(len(arr))
        for sub in combinations(arr, size))

for sub in gen:
    if sum(sub) == 42:
        print(sub)
```

Pourquoi ne PAS utiliser le brute-force ?

Problème d'**explosion combinatoire** (*combinatorial explosion*)

→ L'espace de recherche est très souvent beaucoup plus grand que l'entrée du problème

Sur l'exemple précédent, pour une liste de taille n , on a 2^{n-1} possibilités...

Pour un algorithme en $O(n!)$, avec seulement $n = 30$, l'algorithme continuerait à tourner après la fin de l'univers...



Pourquoi utiliser le brute-force ?

- Pour tester si un algorithme plus complexe est correct
- Parfois il n'y a juste pas mieux.. (décrypter un mot de passe)
- Parfois, même si c'est lent, ça suffit





Backtracking

Backtracking

Introduction

→ Objectif de réduire l'espace de recherche

Exemple : résoudre un Sudoku

- On essaie de remplir les cases (ligne par ligne, colonne par colonne)
- À chaque fois que l'on rencontre un problème, on change le dernier chiffre
- Si aucun chiffre fonctionne, on le supprime et revient au précédent

5	3	4	6	7	8	9	1	2
6	2	7	1	9	5	3	4	8
1	9	8	3	4	2	5	6	7
8	5	9	1	6				3
4			8		3			1
7				2				6
	6					2	8	
			4	1	9			5
				8			7	9

Backtracking

Le principe

- Construire de manière incrémentale l'espace de recherche
- Supprimer les éléments de réponse incorrect (et toutes les autres solutions qui auraient pu contenir cet élément incorrect)

Pour le Sudoku, l'espace est de 9^n mais beaucoup de candidats peuvent être éliminés facilement



Backtracking

Implémentation de la solution pour résoudre le Sudoku



```
def is_valid(grid, i, j, val):  
    line = grid[i]  
    column = [grid[k][j] for k in range(9)]  
    square = [grid[3 * (i // 3) + k][3 * (j // 3) + l]  
              for k in range(3) for l in range(3)]  
    return not (val in line or val in column or val in square)  
  
def backtracking(grid, i, j):  
    if i == 9: return True  
    nexti, nextj = (i if j < 8 else i + 1), (j + 1) % 9  
    if grid[i][j] != 0:  
        return backtracking(grid, nexti, nextj)  
    for val in range(1, 10):  
        if is_valid(grid, i, j, val):  
            grid[i][j] = val  
            if backtracking(grid, nexti, nextj): return True  
            grid[i][j] = 0  
    return False
```



```
grid = [list(map(int, input().split()))  
        for _ in range(9)]  
  
if not backtracking(grid, 0, 0):  
    print("Impossible")  
else:  
    print("\n".join(" ".join(  
        map(str, grid[i])  
        for i in range(9)))
```

Backtracking

Résultat

Entrée

5	3			7				
6			1	9	5			
	9	8					6	7
8				6				3
4			8		3			1
7				2				6
	6					2	8	
			4	1	9			5
				8			7	9

Sortie

5	3	4	6	7	8	9	1	2
6	7	2	1	9	5	3	4	8
1	9	8	3	4	2	5	6	7
8	5	9	7	6	1	4	2	3
4	2	6	8	5	3	7	9	1
7	1	3	9	2	4	8	5	6
9	6	1	5	3	7	2	8	4
2	8	7	4	1	9	6	3	5
3	4	5	2	8	6	1	7	9

Seulement **6428** candidats explorés, au lieu de $8,86293812 \times 10^{21}$

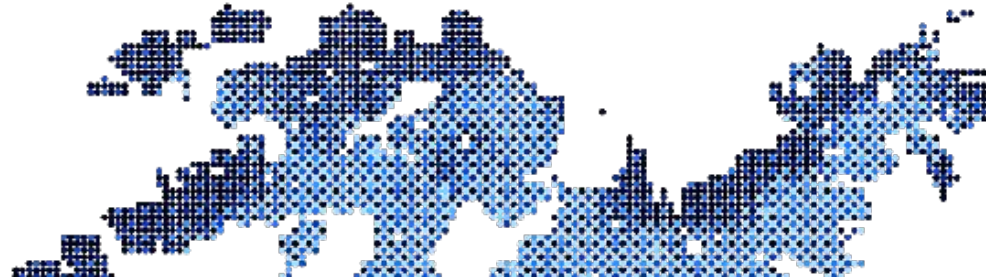


Récurtivité

La récursivité, c'est quoi ?

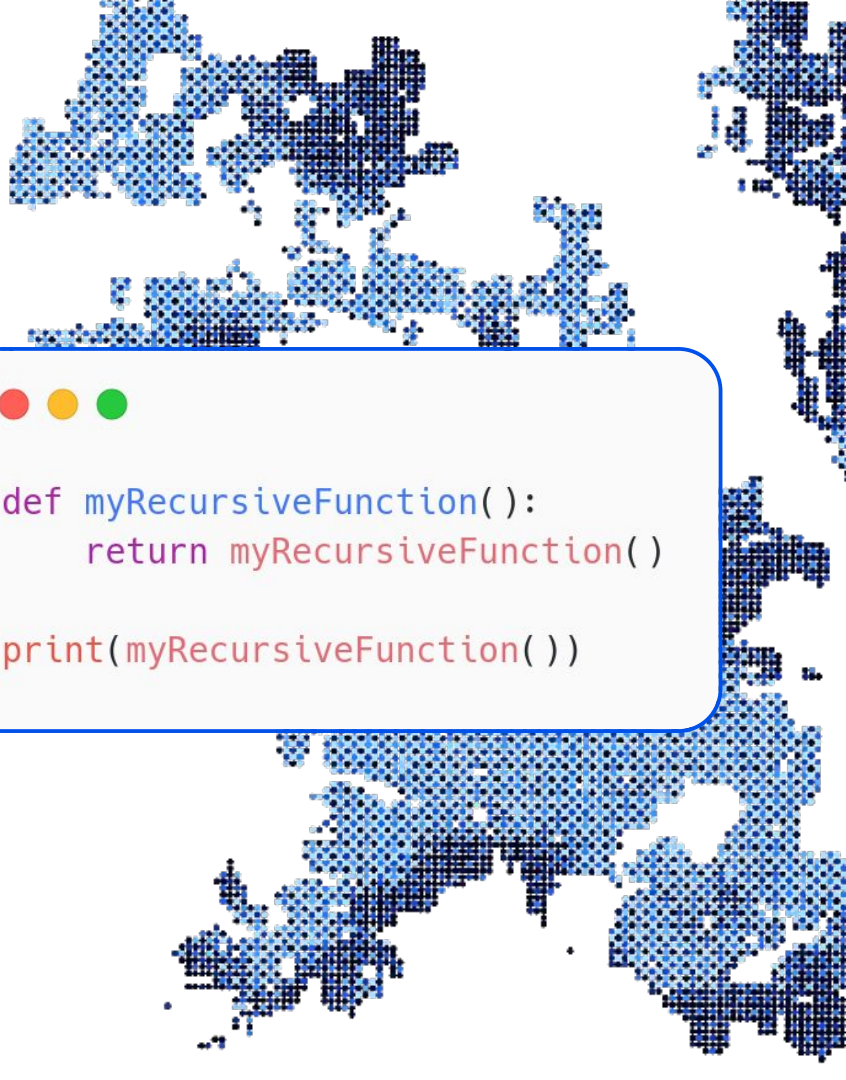
- Méthode pour résoudre des problèmes en utilisant une fonction qui s'appelle elle-même
- Ce code montre ce qu'est la récursivité mais il a plusieurs problèmes.. lesquels ?

```
def myRecursiveFunction():  
    return myRecursiveFunction()  
  
print(myRecursiveFunction())
```



La récursivité, c'est quoi ?

- **Problème 1** : pas de paramètre
 - La fonction appelle une version identique d'elle-même
 - Impossible de changer le fonctionnement de la fonction
- **Problème 2** : pas de **condition d'arrêt**



```
def myRecursiveFunction():  
    return myRecursiveFunction()  
  
print(myRecursiveFunction())
```

Quand l'utiliser ?

- Lorsque le problème principal se divise en sous-problèmes
- On *pourrait* l'utiliser presque partout :
 - Itération $\Leftrightarrow$ Récursivité

Alors pourquoi on l'utilise ?

→ Pour combiner **plusieurs** sous-problèmes pour obtenir la solution



Un premier exemple

La suite de Fibonacci

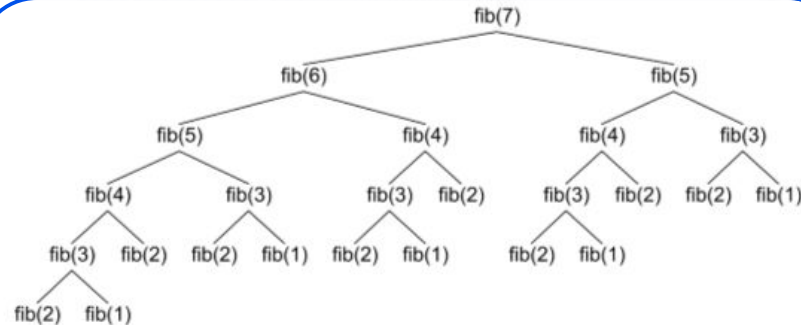
Suite de Fibonacci :

- $F_0 = 0$
- $F_1 = 1$
- $F_2 = F_0 + F_1 = 0 + 1 = 1$
- $F_3 = F_1 + F_2 = 1 + 1 = 2$
- $F_n = F_{n-1} + F_{n-2}$

Objectif : pour trouver F_n , on doit trouver les deux termes précédents F_{n-1} et F_{n-2}

Donc on peut trouver F_n en le divisant en sous problèmes

Cela fonctionne, mais attention à la complexité ! Ici c'est $O(2^n)$



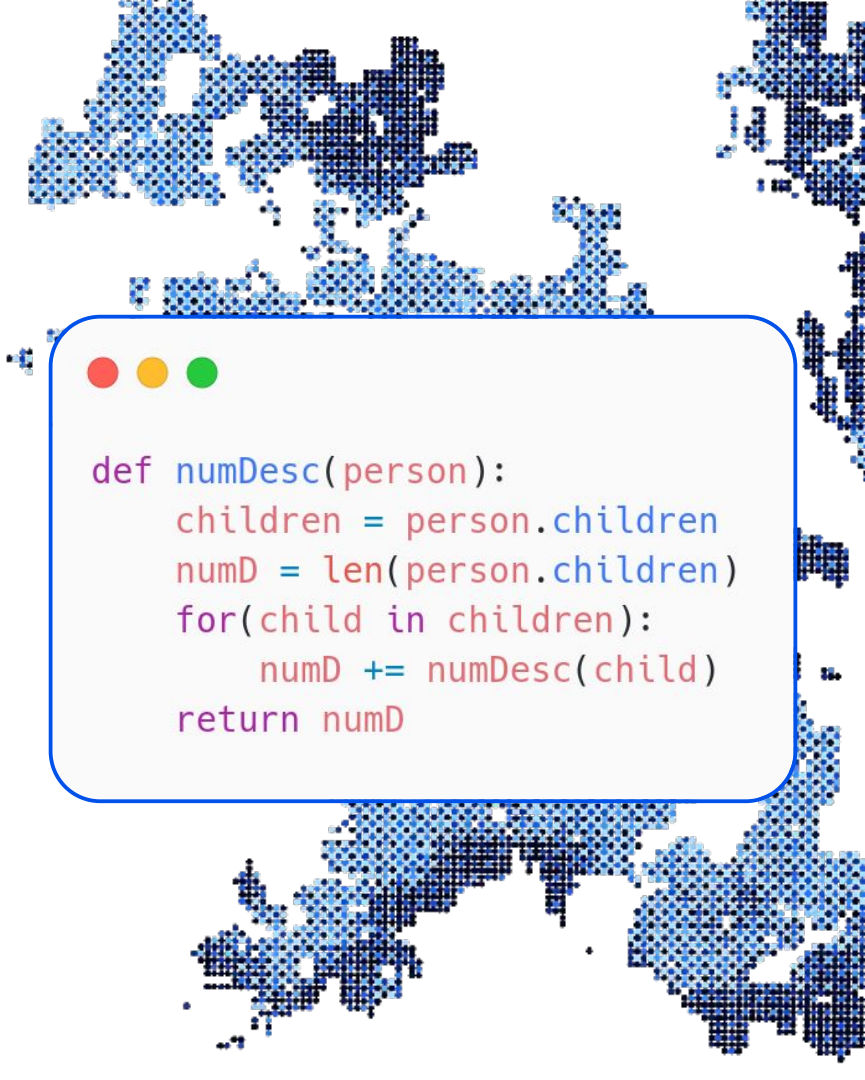
```
def fibonacci(n):  
    if n==0: return 0  
    if n==1: return 1  
    return fibonacci(n-1)+fibonacci(n-2)  
  
print(fibonacci(7))
```

Un deuxième exemple

Le nombre de descendants

À partir d'un arbre généalogique, on veut calculer le nombre de descendants d'une personne

→ Cela revient à calculer la somme des enfants de cette personne, ET le nombre de descendants de chaque enfant

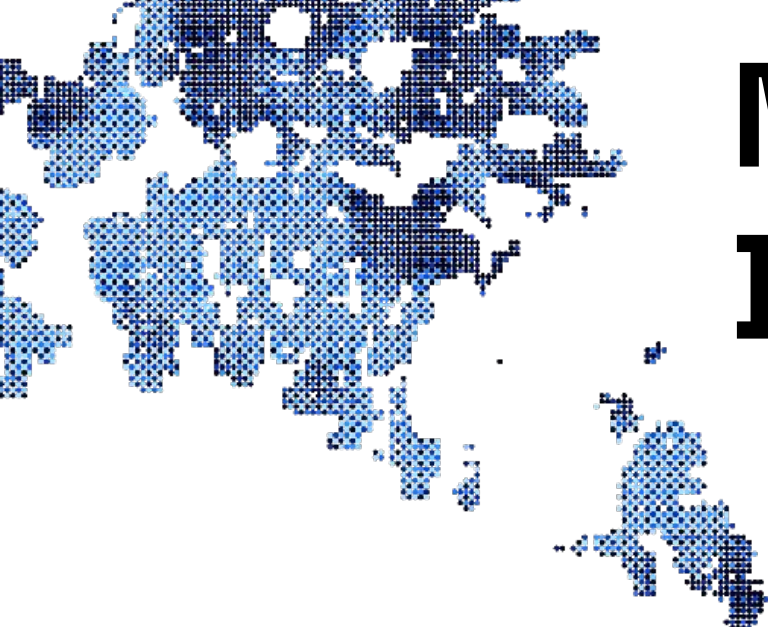


```
def numDesc(person):  
    children = person.children  
    numD = len(children)  
    for child in children:  
        numD += numDesc(child)  
    return numD
```


Comment créer une fonction récursive ?

- Créer une fonction
- Diviser le problème principal en sous-problèmes
- Identifier comment arrêter la récursivité (condition d'arrêt, souvent les cas basiques ou l'impossibilité de calculer davantage)
- Porter attention au résultat renvoyé





Merci !

Des questions ?

Passons aux exercices :D

Crédits (contenu et slides) : Louis Sugy, Arthur Tondereau, William Michaud, Pierre Weisse, Sébastien Goll, Nicolas Foussard

